Practical 1: Weather Data Analysis using OpenWeatherMap API

Aim: Fetch live weather data of 5 Indian cities from a Web API (OpenWeatherMap) using the requests library, convert the JSON response into a pandas DataFrame, compute basic statistics and visualise it with Matplotlib and Seaborn.

Part A: Code explained line by line, with diagrams

Cell 1: Importing libraries

Code line	Explanation
`import os`	Imports Python's os module (file/environment access). It is not actually used later; it is usually kept for reading API keys from environment variables.
`import requests`	Imports the requests library, used to send HTTP GET requests to the web API.
`import pandas as pd`	Imports pandas (alias pd) to build and analyse the tabular DataFrame.
`import matplotlib.pyplot as plt`	Imports pyplot, Matplotlib's plotting interface (alias plt).
`import seaborn as sns`	Imports seaborn, a statistical plotting library built on Matplotlib; used here for the heatmap.
`plt.rcParams["figure.figsize"] = (10, 5)`	Sets the default figure size for every plot to 10 x 5 inches.
`plt.rcParams["axes.grid"] = True`	Turns on grid lines in every plot by default.

Cell 2: Fetching data from the API and creating the DataFrame

Code line	Explanation
`API_KEY = "<your key>"`	Stores the personal API key that authenticates the request (the real key is hidden here; see the security note at the top).
`cities = ["Pune", "Mumbai", "Delhi", "Bangalore", "Chennai"]`	List of the cities whose weather will be fetched.
`weather_list = []`	Empty list; one dictionary (one row) per city will be appended to it.
`for city in cities:`	Loops over every city name.

Code line	Explanation
`url = f"https://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}&units=metric"`	Builds the API URL with an f-string. q = city name, appid = key, units=metric gives temperature in degrees Celsius.
`response = requests.get(url, timeout=15)`	Sends an HTTP GET request. timeout=15 stops the program from waiting more than 15 seconds.
`if response.status_code == 200:`	Status code 200 means the request succeeded. Anything else is handled in the else branch.
`data = response.json()`	Converts the JSON text of the response into a Python dictionary.
`weather_list.append({ ... })`	Adds one dictionary to the list containing the fields below.
`"City": city`	City name from our own list.
`"Temperature": data["main"]["temp"]`	Temperature in degrees C, read from the nested 'main' object of the JSON.
`"Humidity": data["main"]["humidity"]`	Relative humidity in percent.
`"Wind_Speed": data["wind"]["speed"]`	Wind speed in metres per second.

Code line	Explanation
`"Precipitation": data.get("rain", {}).get("1h", 0)`	Rain in the last hour in mm. The 'rain' key exists only when it is raining, so .get() with default {} and then default 0 avoids a KeyError.
`"Latitude" / "Longitude": data["coord"][...]`	Geographic coordinates of the city from the 'coord' object.
`else: print("Error retrieving data for", city, response.status_code)`	If the call fails (wrong key = 401, city not found = 404), prints an error and the loop continues.
`df = pd.DataFrame(weather_list)`	Converts the list of dictionaries to a DataFrame: 5 rows x 7 columns.
`df.fillna(0, inplace=True)`	Data cleaning: any missing value is replaced by 0, in the same DataFrame (inplace).
`print("\nWeather Data:") / display(df)`	Prints a heading and shows the DataFrame as a table.

Cell 3: Descriptive statistics

Code line	Explanation
`df["Temperature"].mean()`	Average temperature of the 5 cities = 27.03 C.
`df["Temperature"].max() / .min()`	Hottest = 30.99 C (Mumbai), coolest = 24.05 C (Delhi).
`df["Humidity"].mean()`	Average humidity = 65.2 %.
`df["Wind_Speed"].mean()`	Average wind speed = 2.594 m/s.

Cell 4: Temperature bar chart

Code line	Explanation
`plt.figure(figsize=(8, 5))`	Creates a new figure of size 8 x 5 (overrides the default).
`plt.bar(df["City"], df["Temperature"])`	Draws a vertical bar chart: x = city, height = temperature.
`plt.xlabel(...) / plt.ylabel(...) / plt.title(...)`	Add the x-axis label, y-axis label and title.
`plt.xticks(rotation=45)`	Rotates the city names by 45 degrees so they do not overlap.
`plt.tight_layout()`	Adjusts spacing so labels are not cut off.
`plt.show()`	Renders the figure.

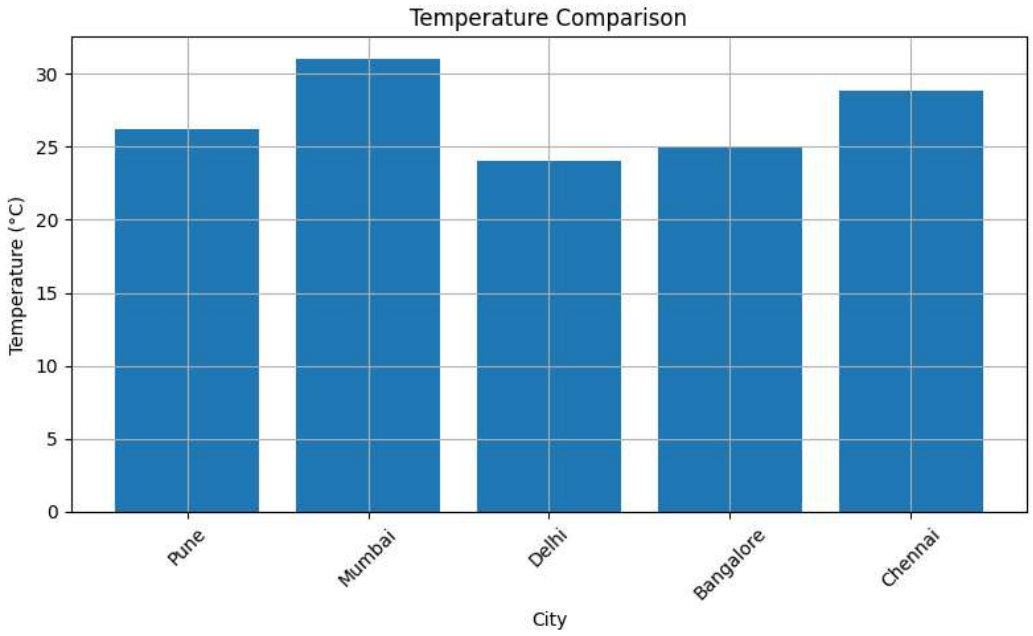


Figure 1: Temperature comparison (bar chart)

What the diagram represents: A bar chart compares one numeric value across categories. Each bar is a city and its height is the temperature in degrees C. Mumbai is the hottest (about 31 C), followed by Chennai (about 29 C), Pune (about 26 C), Bangalore (about 25 C) and Delhi, the coolest (about 24 C). It answers: which city is hottest right now?

Cell 5: Humidity bar chart

Code line	Explanation
`plt.bar(df["City"], df["Humidity"])`	Same as the previous cell but the bar height is humidity (%). All other lines (labels, rotation, layout) work identically.

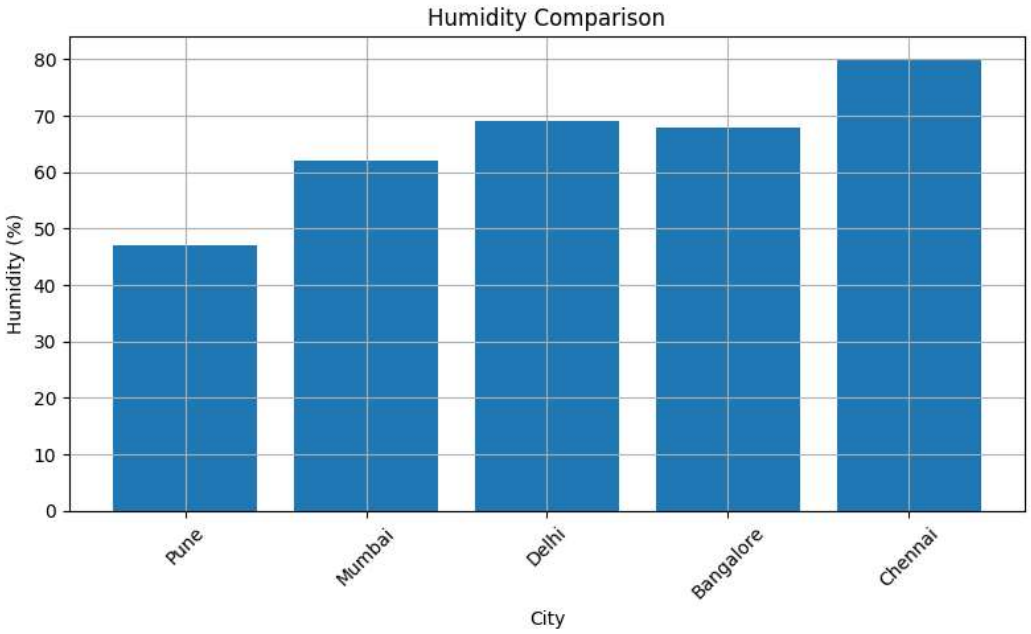


Figure 2: Humidity comparison (bar chart)

What the diagram represents: Chennai has the highest humidity (80 %), then Delhi (69 %) and Bangalore (68 %), Mumbai (62 %) and Pune is the driest (47 %). Coastal Chennai being the most humid is what we expect geographically.

Cell 6: Temperature vs Humidity scatter plot

Code line	Explanation
`plt.scatter(df["Temperature"], df["Humidity"])`	Draws one dot per city: x = temperature, y = humidity. A scatter plot shows the relationship between two numeric variables.

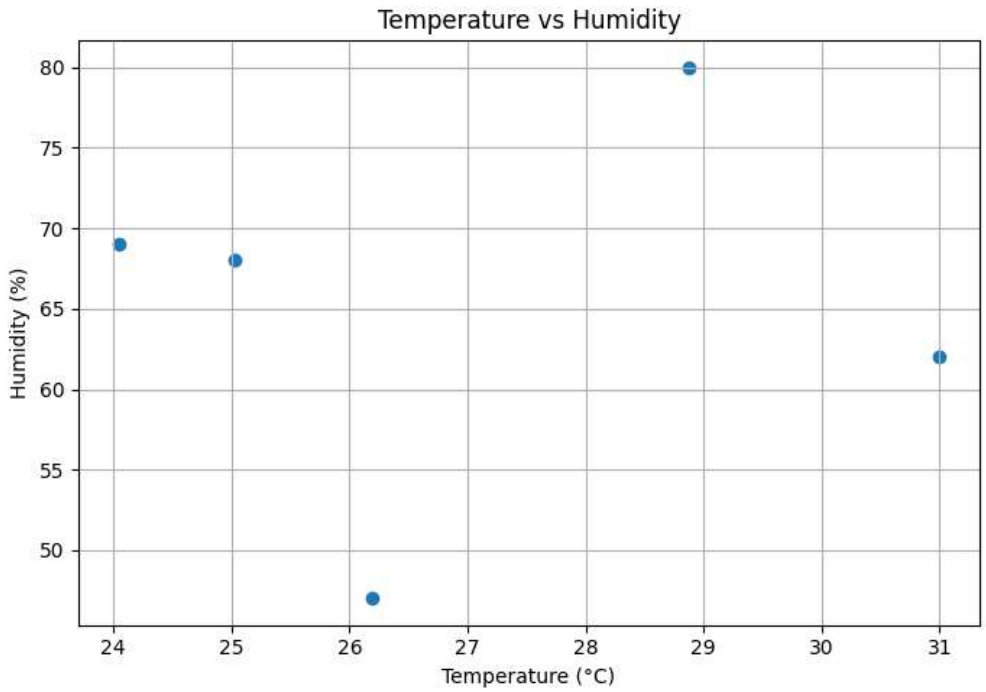


Figure 3: Temperature vs Humidity (scatter plot)

What the diagram represents: Each of the 5 dots is a city. The dots do not form a clear line, so there is no strong relationship between temperature and humidity in this snapshot (correlation is only 0.095, shown in the heatmap). Note: with only 5 points no trend can be called reliable.

Cell 7: Correlation heatmap

Code line	Explanation
`corr = df[["Temperature","Humidity","Wind_Speed","Precipitation"]].corr()`	Selects 4 numeric columns and computes the Pearson correlation matrix (values from -1 to +1).
`sns.heatmap(corr, annot=True, cmap="coolwarm")`	Draws the matrix as a colour grid. annot=True prints the number in every cell; coolwarm colour map: red = positive, blue = negative.

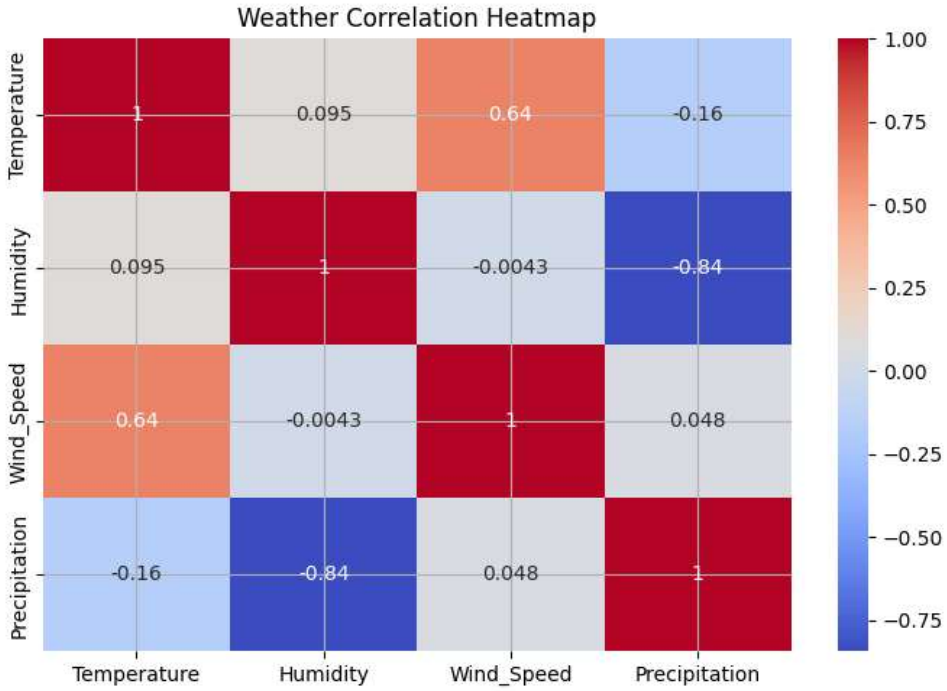


Figure 4: Weather correlation heatmap

What the diagram represents: The diagonal is always 1 (a variable with itself). Strongest relationships: Humidity vs Precipitation = -0.84 (strong negative: higher humidity went with less rain in this tiny sample) and Temperature vs Wind speed = +0.64 (moderate positive). Others are near zero. Correlation does not prove causation, and 5 rows is too small to be conclusive.

Cell 8: Geographical plot

Code line	Explanation
`plt.scatter(df["Longitude"], df["Latitude"], s=100)`	Plots cities as points with longitude on x and latitude on y; s=100 is the marker size.
`for i in range(len(df)):`	Loops over each row index.
`plt.text(df["Longitude"].iloc[i], df["Latitude"].iloc[i], df["City"].iloc[i])`	Writes the city name next to its dot. .iloc[i] picks the i-th row.

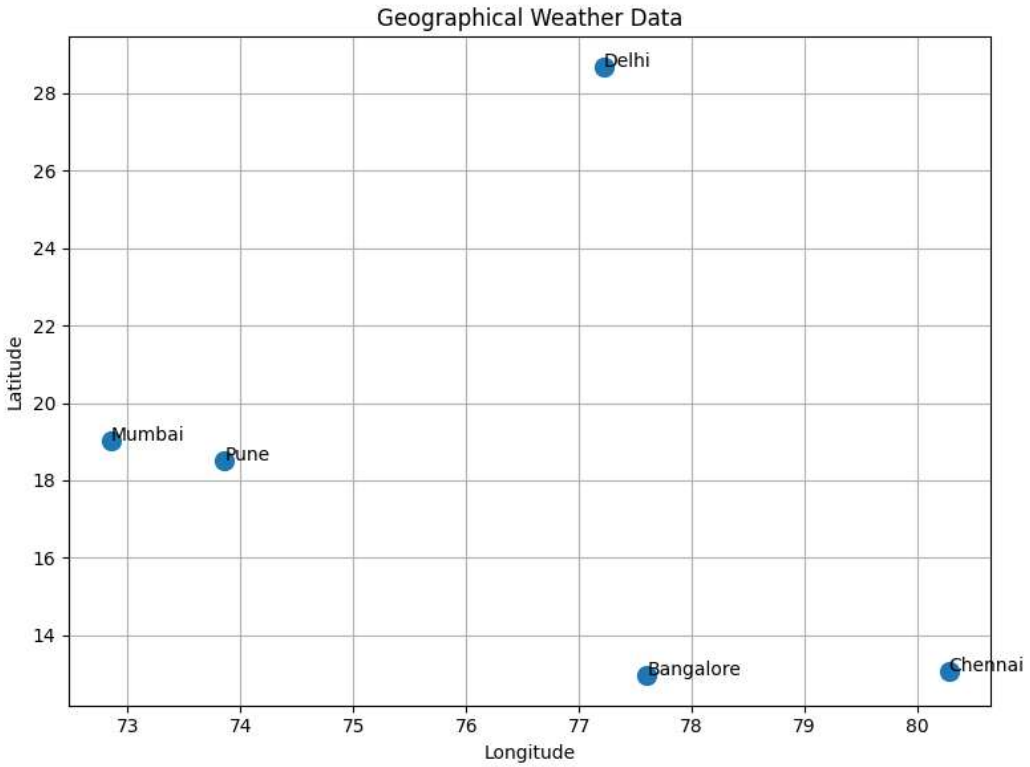


Figure 5: Geographical distribution of the cities

What the diagram represents: A simple map-like plot. Delhi is far north (latitude about 28.7), Mumbai and Pune lie close together on the west side (longitude about 73), Bangalore and Chennai are in the south, with Chennai the furthest east (longitude about 80). It shows where each city sits, which helps relate weather to location.

Part B: Top 5 questions an examiner can ask

Q1. What is an API and what does a REST API return?

Ans: An API (Application Programming Interface) lets one program request data or services from another. A REST API is accessed with HTTP methods such as GET and usually returns data in JSON format, which Python converts to a dictionary with response.json().

Q2. What does status code 200 mean and what other codes can occur?

Ans: 200 means OK (success). 401 means unauthorised (wrong or missing API key), 404 means not found (invalid city) and 429 means too many requests (rate limit exceeded). The code checks for 200 before reading the data.

Q3. Why did you use data.get("rain", {}).get("1h", 0)?

Ans: The 'rain' key is present in the JSON only when it is raining. Using data["rain"] would raise a KeyError, so .get() supplies a safe default (empty dict, then 0 mm).

Q4. What is correlation and how do you read the heatmap?

Ans: Correlation measures the linear relationship between two variables from -1 to +1. +1 is a perfect positive relation, -1 a perfect negative one and 0 none. In the heatmap red cells are positive, blue are negative, and the diagonal is always 1. Here Humidity vs Precipitation is -0.84.

Q5. What does units=metric do, and why use timeout in requests.get()?

Ans: units=metric makes the API return temperature in Celsius (default is Kelvin). timeout prevents the program from hanging forever if the server does not respond. Also mention: the API key should never be hard-coded or shared; store it in an environment variable (that is why os is imported).

Practical 2: Telecom Customer Churn Data Preprocessing

Aim: Clean and prepare the Telecom Customer Churn dataset (7043 customers, 21 columns) for machine learning: explore it, handle missing values, standardise text, fix data types, treat outliers, encode categorical variables, scale features and split into training and testing sets.

Part A: Code explained line by line, with diagrams

Cell 1: Importing libraries

Code line	Explanation
`import pandas as pd`	pandas for data handling.
`import numpy as np`	NumPy for numeric helpers (np.number is used to select numeric columns).
`from sklearn.model_selection import train_test_split`	Function that splits data randomly into training and testing parts.
`from sklearn.preprocessing import StandardScaler`	Class that standardises features to mean 0 and standard deviation 1.

Cell 2: Loading and first look at the data

Code line	Explanation
`df = pd.read_csv("telecom_customer_churn.csv")`	Reads the CSV file into a DataFrame called df.
`display(df.head())`	Shows the first 5 rows to see the columns and sample values.
`print(df.shape)`	Prints (rows, columns) = (7043, 21).

Cells 3-4: Structure and data types

Code line	Explanation
`df.info()`	Summarises column names, non-null counts and dtypes plus memory usage. Quickly shows if any column has missing values.
`print(df.dtypes)`	Lists only the data type of every column (object = text, int64/float64 = numbers).

Cells 5-7: Exploring the dataset

Code line	Explanation
`df.describe()`	Statistics for the numeric columns: count, mean, std, min, quartiles, max. TotalCharges has count 7032, i.e. 11 values are missing.
`df.isnull().sum()`	Counts missing (NaN) values in every column.
`df.duplicated().sum()`	Counts fully duplicated rows. Result is 0, so nothing needs removing.

Cell 8: Handling missing values

Code line	Explanation
`numeric_cols = df.select_dtypes(include=np.number).columns`	Gets the names of all numeric columns.
`for col in numeric_cols:`	Loop over each numeric column.
`df[col] = df[col].fillna(df[col].median())`	Fills missing numbers with the column median. The median is used because it is not affected by outliers.

Code line	Explanation
`categorical_cols = df.select_dtypes(include='object').columns`	Gets the names of all text columns.
`if df[col].isnull().sum() > 0:`	Acts only on text columns that actually have missing values.
`df[col] = df[col].fillna(df[col].mode()[0])`	Fills missing text with the mode (most frequent value). mode() returns a Series, so [0] takes the first.
`print(df.isnull().sum())`	Confirms that all missing values are now 0.

Cell 9: Standardising text values

Code line	Explanation
`for col in df.select_dtypes(include='object').columns:`	Loop over every text column.
`df[col] = df[col].astype(str).str.strip().str.lower()`	Converts to string, removes leading/trailing spaces and converts to lower case, so 'Yes', ' yes' and 'YES' all become 'yes'. This is why customerID and values appear in lower case in the output.

Cell 10: Fixing data types

Code line	Explanation
`df["customerID"] = df["customerID"].astype(str)`	Makes sure the identifier is stored as text.
`numeric_columns = ["SeniorCitizen","tenure","MonthlyCharges","TotalCharges"]`	Columns that must be numeric.
`df[col] = pd.to_numeric(df[col], errors="coerce")`	Converts to numbers. errors='coerce' turns any value that cannot be converted (e.g. a blank space) into NaN instead of raising an error.

Cell 11: Outlier treatment (IQR method)

Code line	Explanation
`if col == "SeniorCitizen": continue`	Skips this column because it is a 0/1 flag, not a real measurement.
`Q1 = df[col].quantile(0.25) / Q3 = df[col].quantile(0.75)`	First quartile (25th percentile) and third quartile (75th percentile).
`IQR = Q3 - Q1`	Inter-quartile range: the spread of the middle 50 % of the data.
`lower_limit = Q1 - 1.5 * IQR / upper_limit = Q3 + 1.5 * IQR`	Tukey fences. Values outside these limits are treated as outliers.
`df[col] = df[col].clip(lower_limit, upper_limit)`	Winsorising: values below the lower limit are set to the lower limit and values above the upper limit to the upper limit. Rows are kept; only extreme values are capped.

Cells 12-14: Dropping columns and one-hot encoding

Code line	Explanation
`df_model = df.drop(columns=["customerID", "Churn"])`	Removes customerID (just an identifier, no predictive value) and Churn (the target; it must be kept separate from the features).
`df_encoded = pd.get_dummies(df_model, drop_first=True, dtype=int)`	One-hot encoding: each text column becomes 0/1 columns. drop_first=True removes one dummy per column to avoid the dummy-variable trap (multicollinearity). dtype=int gives 0/1 instead of True/False. Result shape (7043, 30).

Cell 15: Target variable

Code line	Explanation
`print(df["Churn"].value_counts())`	Shows class balance: 5174 'no' and 1869 'yes' (about 73 % / 27 %), so the data is imbalanced.
`df_encoded["Churn"] = df["Churn"].str.lower().map({"yes":1, "no":0})`	Converts the target to numbers (yes = 1, no = 0) and adds it back to the encoded data.
`X = df_encoded.drop(columns=["Churn"])`	X = feature matrix (30 columns).
`y = df_encoded["Churn"]`	y = target vector (what we want to predict).

Cells 16-17: Scaling and train/test split

Code line	Explanation
`scaler = StandardScaler()`	Creates the scaler object.
`X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)`	fit_transform learns each column's mean and standard deviation and applies z = (x - mean)/std. The result is a NumPy array, so it is wrapped back into a DataFrame with the original column names.
`X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.20, random_state=42)`	Randomly splits the data: 80 % training (5634 rows) and 20 % testing (1409 rows). random_state=42 makes the split reproducible.
`print(X_train.shape) / print(X_test.shape)`	Prints (5634, 30) and (1409, 30).

Cell 18: Exporting

Code line	Explanation
`df.to_csv("Telecom_Customer_Churn_Cleaned.csv", index=False)`	Saves the cleaned dataset. index=False avoids writing the row numbers as an extra column.
`df_encoded.to_csv("Telecom_Customer_Churn_Encoded.csv", index=False)`	Saves the encoded dataset (features + Churn) ready for modelling.

Workflow diagram

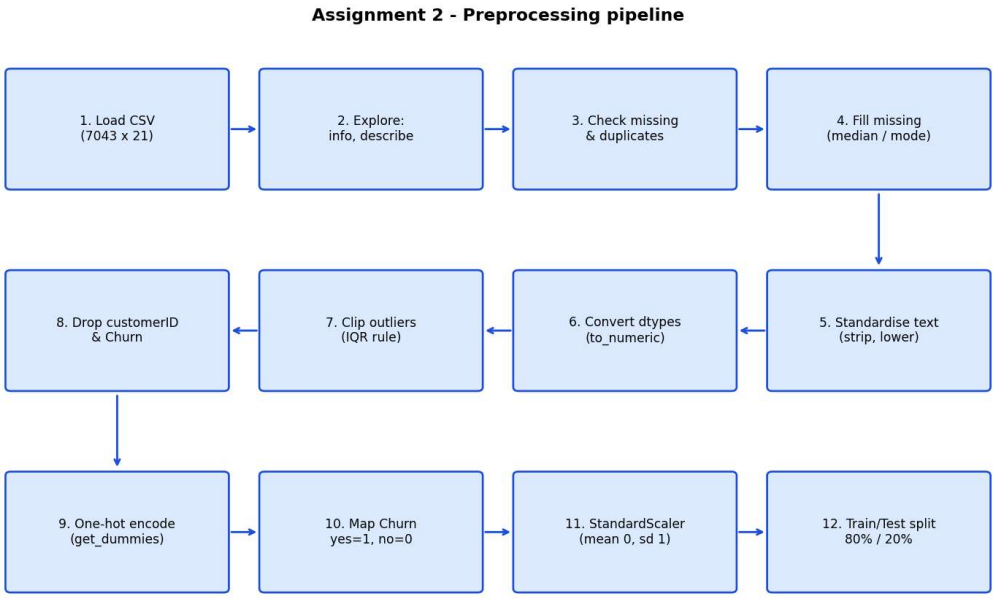


Figure 6: Preprocessing workflow (diagram added to summarise the practical)

What the diagram represents: This practical produces tables, not plots, so this flow diagram shows what the notebook does, step by step: load, explore, check missing values and duplicates, fill missing values, standardise text, convert types, cap outliers, drop identifier and target, one-hot encode, map the target, scale and finally split into train and test sets. Each box matches a cell of the notebook, and the order matters: cleaning comes before encoding and scaling.

Part B: Top 5 questions an examiner can ask

Q1. Why do we preprocess data? What are the main steps you performed?

Ans: Real data is dirty (missing values, wrong types, inconsistent text, outliers) and ML models need clean numeric input. Steps: explore, handle missing values, remove duplicates (none found), standardise text, fix data types, treat outliers (IQR), drop the ID and target, one-hot encode, scale, and train/test split.

Q2. Why median for numeric and mode for categorical missing values?

Ans: Median is robust to outliers (unlike mean), so it is a safer central value for numbers. Mode, the most frequent value, is the only meaningful central value for text categories. Only TotalCharges had missing values (11 rows) in this dataset.

Q3. Explain the IQR method. Why clip instead of delete outliers?

Ans: IQR = Q3 - Q1. Values below Q1 - 1.5 x IQR or above Q3 + 1.5 x IQR are outliers. Clipping (winsorising) caps them at the limit, which keeps all 7043 customers so no information or class balance is lost.

Q4. What is one-hot encoding and why drop_first=True?

Ans: Machine-learning models need numbers, so each category becomes a separate 0/1 column. If we keep all k dummies, one can be predicted from the others (dummy-variable trap / multicollinearity), so one is dropped (k-1 columns). Here 21 columns became 30 features.

Q5. What is StandardScaler and what is the train/test split? Any drawback in your code?

Ans: StandardScaler transforms each feature to mean 0 and standard deviation 1, which helps distance- and gradient-based models (KNN, SVM, logistic regression). The split keeps 20 % of data unseen to evaluate the model. Drawback: the scaler was fitted on all data before splitting, which leaks test-set information (data leakage); the correct way is to split first, then fit the scaler on X_train only and transform X_test. Also, the data is imbalanced (73:27), so stratify=y would be better in train_test_split.

Practical 3: Real Estate Data Wrangling

Aim: Wrangle a small real-estate dataset (100 properties, 9 columns): clean column names, fill missing values, merge with another table, filter and subset, encode categories, aggregate prices, and detect and remove outliers.

Part A: Code explained line by line, with diagrams

Cell 1: Imports and loading

Code line	Explanation
`import pandas as pd / import numpy as np`	Data handling and numeric libraries.
`df = pd.read_csv("RealEstate_Prices.csv")`	Loads the real-estate data.
`display(df.head()) / print(df.shape)`	Shows the first 5 rows and the shape (100, 9).

Cell 2: Cleaning column names and missing values

Code line	Explanation
`df.columns = (df.columns ...`	Assigns a cleaned list of names back to the DataFrame. The chain below is applied to every column name.
`.str.strip()`	Removes extra spaces at the start and end.
`.str.lower()`	Converts to lower case.
`.str.replace(" ", "_", regex=False)`	Replaces spaces by underscores, e.g. 'Property ID' becomes 'property_id'.
`.str.replace(r"[^\w]", "", regex=True)`	Regular expression that deletes every character that is not a letter, digit or underscore. This removes brackets and the $ sign, which is why 'Sale Price ($)' became 'sale_price_'.
`df.isnull().sum()`	Missing values: neighborhood = 2, bathrooms = 3.
`df["bathrooms"] = df["bathrooms"].fillna(df["bathrooms"].median())`	Numeric column filled with the median.
`df["neighborhood"] = df["neighborhood"].fillna(df["neighborhood"].mode()[0])`	Categorical column filled with the most frequent neighbourhood.

Cell 3: Merging datasets

Code line	Explanation
`neighborhood_data = pd.DataFrame({...})`	Creates a second small table with each neighbourhood and its average income (Downtown 75000, Suburb 65000, Riverside 60000, Hillview 70000, Lakeside 72000).
`df = pd.merge(df, neighborhood_data, on="neighborhood", how="left")`	Joins the two tables on the common key neighborhood. how='left' keeps all rows of df and adds the avg_income column (NaN if no match). Columns go from 9 to 10.

Cell 4: Filtering and subsetting

Code line	Explanation
`filtered_data = df[(df["sale_year"] >= 2021) & (df["sale_year"] <= 2023)]`	Boolean filtering: keeps properties sold between 2021 and 2023. Each condition needs brackets and & (and) is used instead of the Python word 'and'.
`houses = df[df["property_type"] == "House"]`	Keeps only rows where the type is House.
`subset = df[["property_type","neighborhood","bedrooms","area_sq_ft","sale_price_"]]`	Column selection: double square brackets return a smaller DataFrame with only the important columns.

Cell 5: Handling categorical variables

Code line	Explanation
`df_encoded = pd.get_dummies(df, columns=["property_type","neighborhood"], dtype=int)`	One-hot encodes only the two named columns into 0/1 columns such as property_type_House and neighborhood_Downtown. Other columns are untouched. Here drop_first is not used, so every category keeps its own column.
`print(df_encoded.columns.tolist())`	Prints the new column list to verify the encoding.

Cell 6: Aggregation

Code line	Explanation
`df.groupby("neighborhood")["sale_price_"].mean()`	Splits the data by neighbourhood and averages sale price in each group.
`.sort_values(ascending=False)`	Sorts highest first. Result: Downtown 552,412 > Lakeside 534,054 > Suburb 499,499 > Riverside 462,908 > Hillview 425,439.
`df.groupby("property_type")["sale_price_"].mean()`	Same by property type: House 524,993 > Villa 521,298 > Apartment 467,666 > Townhouse 461,895.
`df["price_per_sq_ft"] = df["sale_price_"] / df["area_sq_ft"]`	Creates a new feature: price per square foot, calculated for every row (vectorised).

Cells 7-8: Outlier detection and removal

Code line	Explanation
`def detect_outliers(data, column):`	Defines a reusable function that works for any column.
`Q1, Q3, IQR, lower_bound, upper_bound`	Computes quartiles and the Tukey fences exactly as in Practical 2.
`outliers = data[(data[column] < lower_bound) | (data[column] > upper_bound)]`	Rows outside the fences; | means OR.
`return outliers, lower_bound, upper_bound`	Returns three values: the outlier rows and the two limits.
`price_outliers, lower, upper = detect_outliers(df, "sale_price_")`	Finds price outliers: property 1081 with price 3,500,000 (upper bound 1,247,386.6).
`area_outliers, lower_area, upper_area = detect_outliers(df, "area_sq_ft")`	Finds area outliers: property 1091 with 12,000 sq ft.

Code line	Explanation
`df_clean = df[(price >= lower) & (price <= upper) & (area >= lower_area) & (area <= upper_area)]`	Keeps only rows within both sets of limits, i.e. removes those 2 rows. Shape changes from (100, 11) to (98, 11).
`df_clean.info()`	Final check of dtypes and non-null counts of the clean 98-row dataset.

Workflow diagram

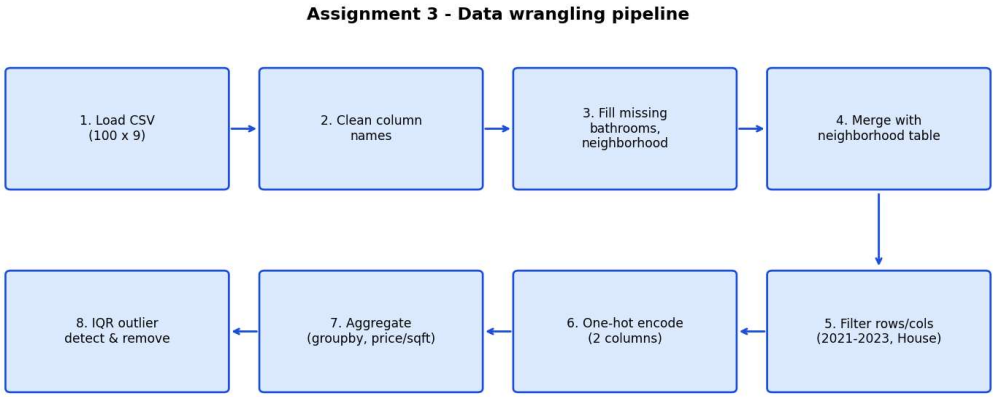


Figure 7: Data wrangling workflow (diagram added to summarise the practical)

What the diagram represents: This practical outputs tables only, so this diagram summarises the process: load, clean column names, fill missing values, merge with the neighbourhood table, filter and subset, one-hot encode, aggregate, and finally detect and remove outliers. The size goes 100 rows to 98 rows at the last step.

Part B: Top 5 questions an examiner can ask

Q1. What is data wrangling? How is it different from data cleaning?

Ans: Data wrangling (munging) is transforming raw data into a usable form: cleaning, merging, filtering, reshaping, encoding and aggregating. Cleaning (fixing missing values, duplicates, errors) is one part of wrangling.

Q2. Explain pd.merge() and the types of joins. Why how='left' here?

Ans: pd.merge combines two DataFrames on a common key. Types: inner (only matching keys), left (all rows of the left table), right, and outer (all rows of both). A left join keeps every property even if its neighbourhood is missing in the income table, so no property is lost.

Q3. What does groupby() do?

Ans: groupby splits the data into groups by a column (split), applies a function such as mean, sum or count to each group (apply), and combines the results (combine). Example: the average sale price per neighbourhood, where Downtown is highest.

Q4. How did you detect and handle outliers? Why remove them here?

Ans: Using the IQR rule: values beyond Q1 - 1.5 IQR or Q3 + 1.5 IQR are outliers. Property 1081 (price 3.5 million) and property 1091 (12,000 sq ft) were extreme and would distort averages and models, so the 2 rows were removed (100 to 98). Mention that an alternative is capping or investigating whether they are data-entry errors.

Q5. Why did the column 'Sale Price ($)' become 'sale_price_'? What are median and mode used for?

Ans: The regex [^\w] removed every character that is not a letter, digit or underscore, so the brackets and $ vanished but the underscore from the space remained. Median fills missing numeric values (bathrooms) robustly, while mode fills missing categorical values (neighborhood).

Practical 4: Data Visualization using Matplotlib (Air Quality Index Trends)

Aim: Analyse the Air Quality Index (AQI) and pollutants (PM2.5, PM10, CO) of a city for 60 days (1 Jan 2025 to 1 Mar 2025) and visualise trends, distribution, monthly comparison and the relationship of pollutants with AQI.

Part A: Code explained line by line, with diagrams

Cell 1: Imports and settings

Code line	Explanation
`import pandas as pd / import matplotlib.pyplot as plt`	pandas for data and pyplot for plotting.
`plt.rcParams["figure.figsize"] = (10, 5)`	Default figure size 10 x 5 inches.
`plt.rcParams["axes.grid"] = True`	Grid on by default.

Cell 2: Loading with error handling

Code line	Explanation
`FILE = "City_Air_Quality.csv"`	Stores the file name in a variable.
`try: df = pd.read_csv(FILE)`	Tries to read the file.
`except FileNotFoundError: print(...)`	Runs only if the file does not exist and prints a friendly message instead of crashing.
`else: print(f"Loaded {FILE}")`	else of try runs only if no exception occurred.
`display(df.head())`	Shows the first 5 rows.

Cell 3: Understanding the dataset

Code line	Explanation
`df.shape`	(60, 5): 60 days, 5 columns.
`df.columns.tolist()`	['Date','PM2.5','PM10','CO','AQI'].
`df.dtypes.to_frame("dtype")`	Data types as a neat one-column table. Date is still 'object' (text).
`df.isna().sum().to_frame("missing")`	Missing values per column: all 0.
`df.describe(include="all").T`	Statistics for all columns; .T transposes so variables are rows. AQI mean = 122.5, min = 81.2, max = 176.3.

Cell 4: Identifying the variables automatically

Code line	Explanation
`date_candidates = [c for c in df.columns if c.lower() in {...} or "date" in c.lower()]`	List comprehension: collects columns whose name looks like a date column.
`aqi_candidates = [c for c in df.columns if "aqi" in c.lower()]`	Collects columns containing 'aqi'.
`date_col = date_candidates[0] if date_candidates else None`	Picks the first match, or None if there is none (conditional expression).
`df[date_col] = pd.to_datetime(df[date_col], errors="coerce")`	Converts the Date text to real datetime values; invalid dates become NaT.

Code line	Explanation
`pollutant_keywords = [...]  / pollutant_cols = [...]`	Finds columns whose names contain pollutant keywords: ['PM2.5', 'PM10', 'CO'].

Cell 5: AQI trend line plot

Code line	Explanation
`plot_df = df.dropna(subset=[date_col, aqi_col]).sort_values(date_col)`	Drops rows with missing date/AQI and sorts by date so the line goes forward in time.
`plt.plot(plot_df[date_col], plot_df[aqi_col], marker="o", linewidth=1.5)`	Line chart: x = date, y = AQI; marker='o' shows a circle at every day; linewidth sets the thickness.
`plt.title / xlabel / ylabel / xticks(rotation=45) / tight_layout / show`	Labels, rotated dates, layout and display.

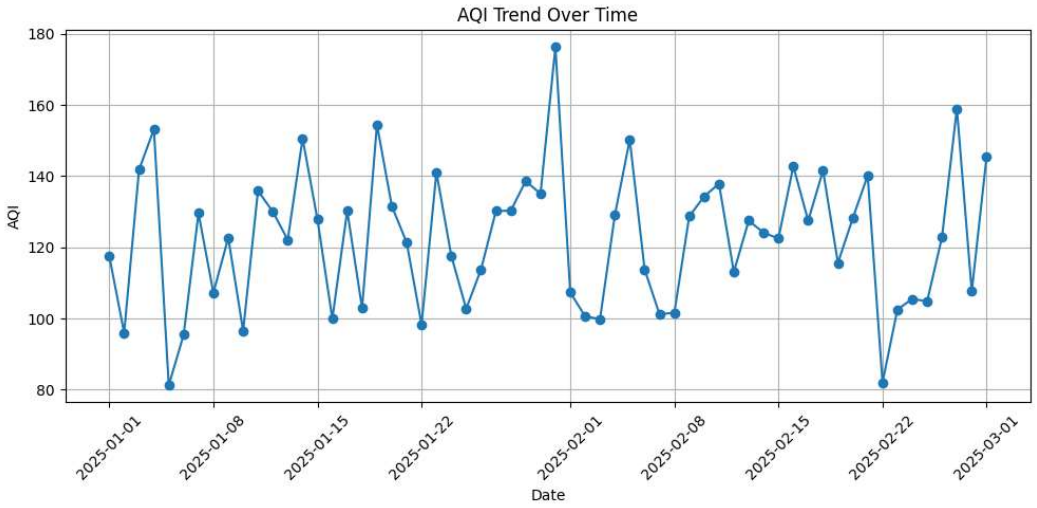


Figure 8: AQI trend over time (line chart)

What the diagram represents: A line chart shows how a value changes over time. AQI fluctuates daily between about 81 and 176. The highest AQI (176.3) occurs on 31 January, and there are dips near 81 in early January and late February. The overall level stays in the 100-150 range, i.e. moderate to poor air quality.

Cell 6: Pollutant trend plots (loop)

Code line	Explanation
`for col in pollutant_cols:`	Repeats the plotting for PM2.5, PM10 and CO, so three charts are made with one block of code.
`if pd.api.types.is_numeric_dtype(plot_df[col]):`	Safety check: plot only numeric columns.
`plt.plot(..., marker=".", label=col)  / plt.legend()`	Line plot with small dots; label= together with legend() shows the pollutant name in a legend box.

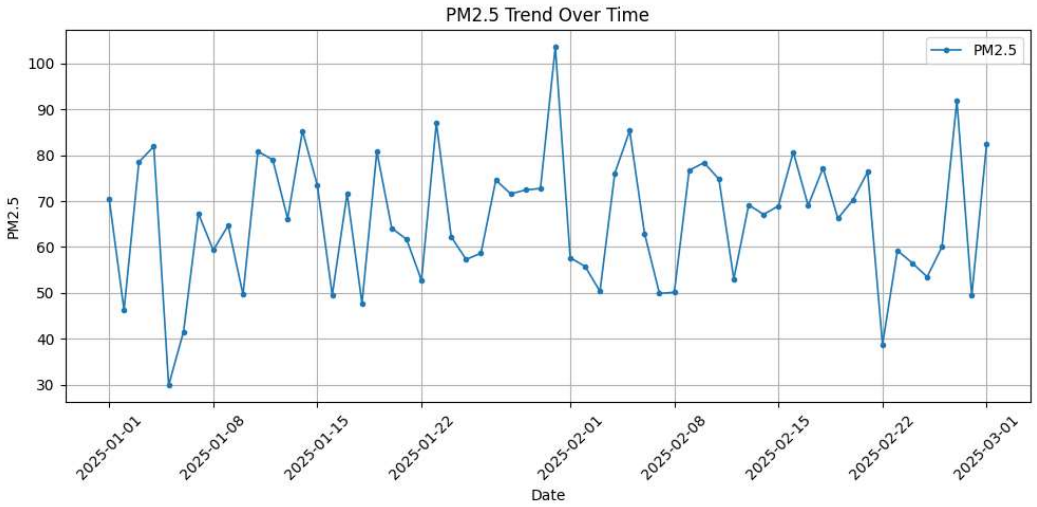


Figure 9: PM2.5 trend over time

What the diagram represents: PM2.5 (fine particles smaller than 2.5 micrometres) moves between about 30 and 103. The sharpest peak (about 103) is on 31 January, exactly the day AQI peaked, which hints that PM2.5 drives AQI.

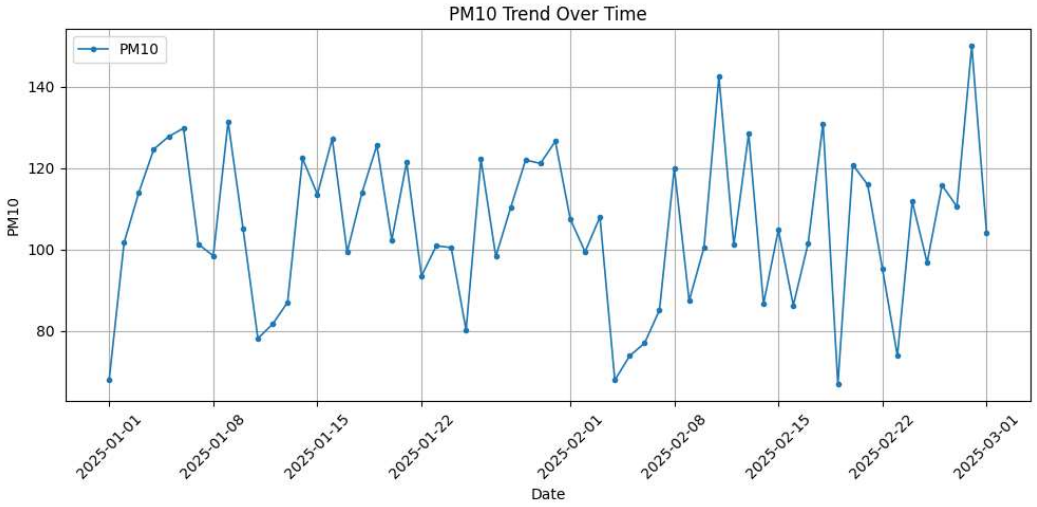


Figure 10: PM10 trend over time

What the diagram represents: PM10 (coarser particles) ranges from about 67 to 150. Its pattern is irregular and its peak (about 150) comes on 28 February, not on the AQI peak day, so it follows AQI less closely.

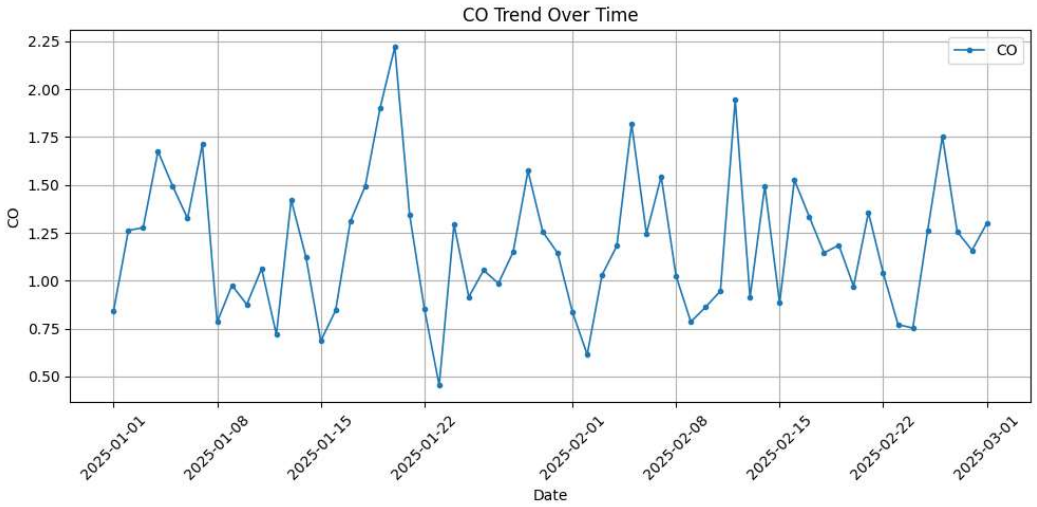


Figure 11: CO trend over time

What the diagram represents: Carbon monoxide ranges from about 0.45 to 2.22. It peaks around 20 January and shows no clear match with AQI peaks.

Cell 7: Average AQI by month (bar chart)

Code line	Explanation
`plot_df["Period"] = plot_df[date_col].dt.to_period("M").astype(str)`	Creates a month label like '2025-01' from each date; .dt gives datetime helpers and to_period('M') converts to month.
`period_aqi = plot_df.groupby("Period")[aqi_col].mean()`	Average AQI for each month.
`period_aqi.plot(kind="bar")`	Draws the bar chart directly from the pandas Series.

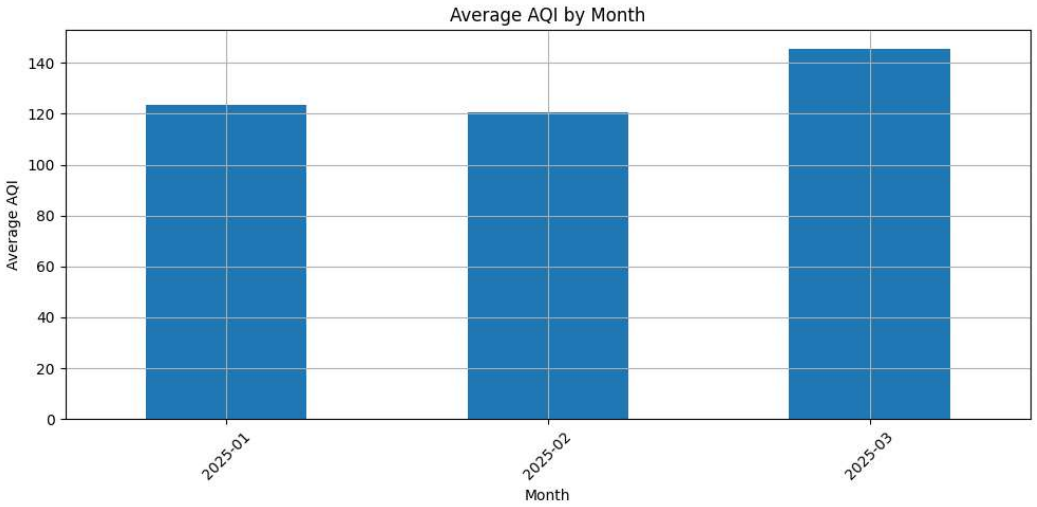


Figure 12: Average AQI by month (bar chart)

What the diagram represents: Compares the monthly average AQI: January about 123.6, February about 120.4 and March about 145.6. Caution for the viva: the dataset ends on 1 March, so the March bar is based on only one day and should not be read as a true monthly average.

Cell 8: Distribution of AQI (boxplot and histogram)

Code line	Explanation
`plt.boxplot(df[aqi_col].dropna(), vert=True)`	Box-and-whisker plot. dropna() removes missing values; vert=True draws it vertically.
`plt.hist(df[aqi_col].dropna(), bins=15, edgecolor="black")`	Histogram: divides AQI into 15 equal intervals (bins) and counts how many days fall in each; edgecolor draws black borders on the bars.

Code line	Explanation
`plt.ylabel("Frequency")`	In a histogram the y-axis is the number of days.

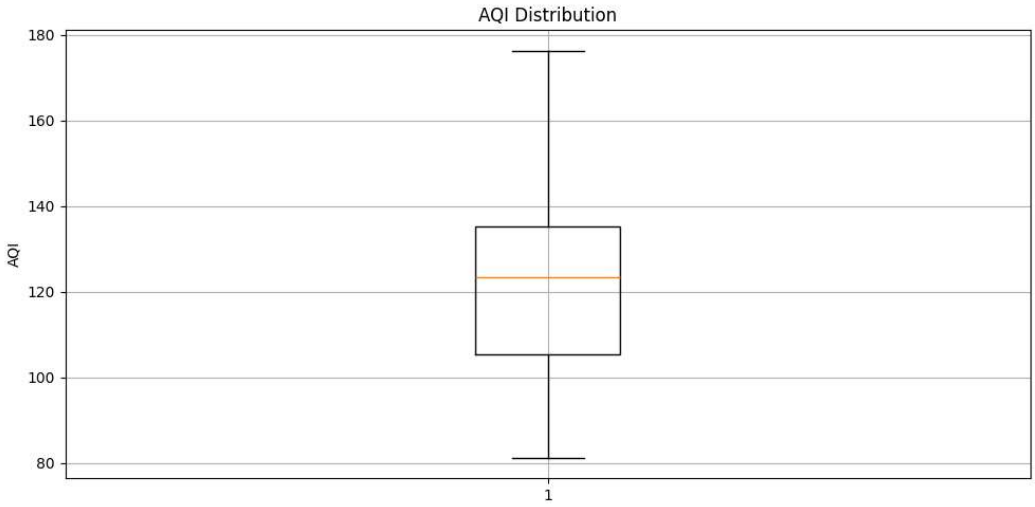


Figure 13: AQI boxplot

What the diagram represents: The box spans Q1 = 105.3 to Q3 = 135.3 (the middle 50 % of days) and the orange line is the median (123.5). The whiskers reach the minimum 81.2 and maximum 176.3. No separate outlier dots appear, so there are no extreme AQI values.

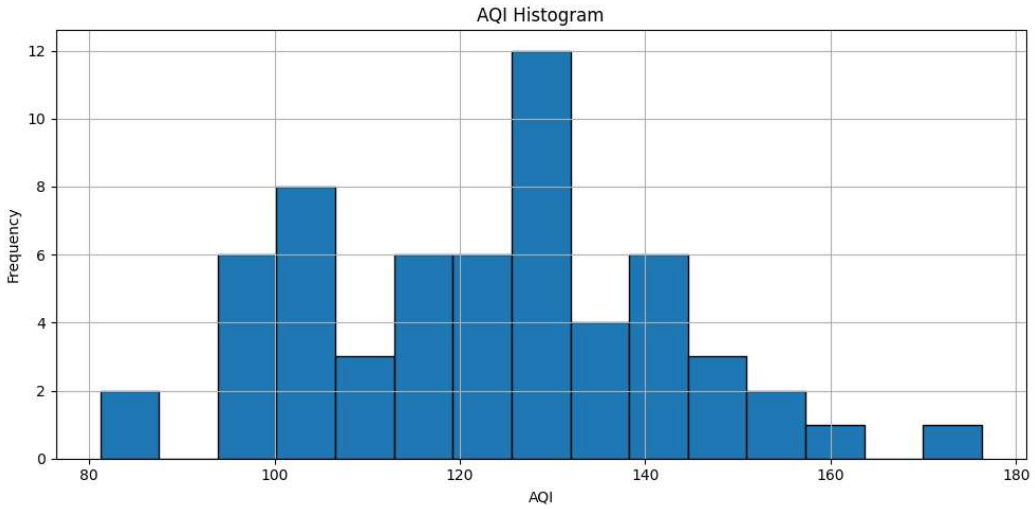


Figure 14: AQI histogram

What the diagram represents: Shows the shape of the AQI distribution. Most days fall between about 95 and 145 with the tallest bar (12 days) near 126-132. The distribution is roughly bell-shaped with a slight tail towards high values.

Cell 9: Scatter plots and correlation (loop)

Code line	Explanation
`temp = df[[aqi_col, col]].dropna()`	Selects AQI and one pollutant and drops rows with missing values.
`plt.scatter(temp[col], temp[aqi_col], alpha=0.7)`	Scatter plot of pollutant (x) against AQI (y); alpha=0.7 makes dots slightly transparent so overlapping points are visible.
`temp[aqi_col].corr(temp[col])`	Pearson correlation coefficient between AQI and the pollutant.
`print(f"... {value:.3f}")`	Prints it rounded to 3 decimal places.

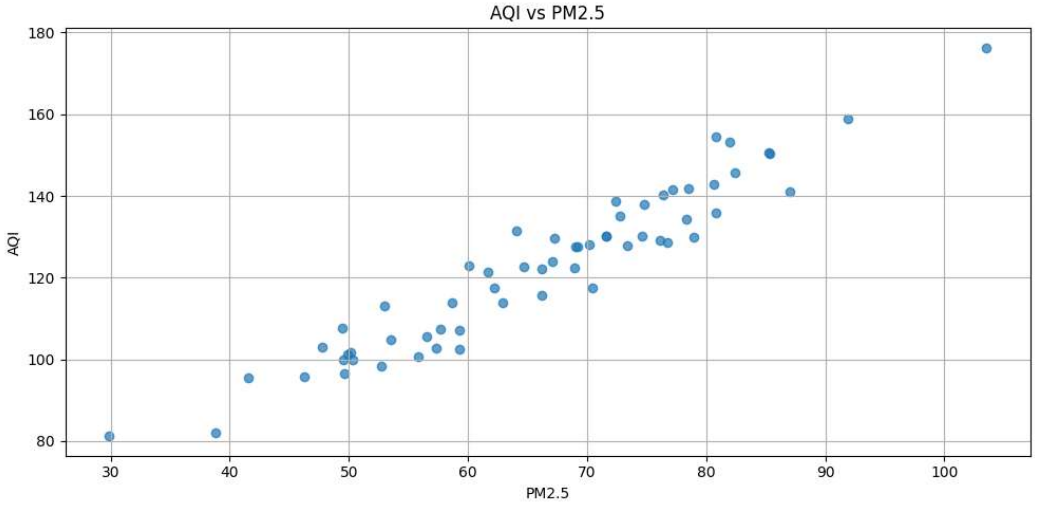


Figure 15: AQI vs PM2.5 (scatter plot), correlation = 0.958

What the diagram represents: The dots form a tight upward line, i.e. a very strong positive relationship: when PM2.5 rises, AQI rises almost proportionally. PM2.5 is the main driver of AQI here.

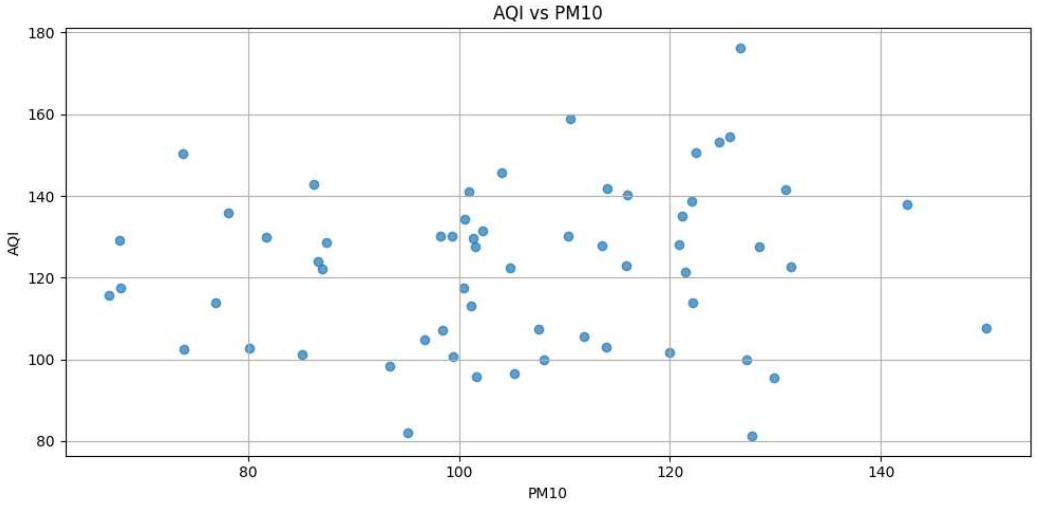


Figure 16: AQI vs PM10 (scatter plot), correlation = 0.117

What the diagram represents: The dots are scattered with no clear pattern, so there is a very weak relationship between PM10 and AQI in this data.

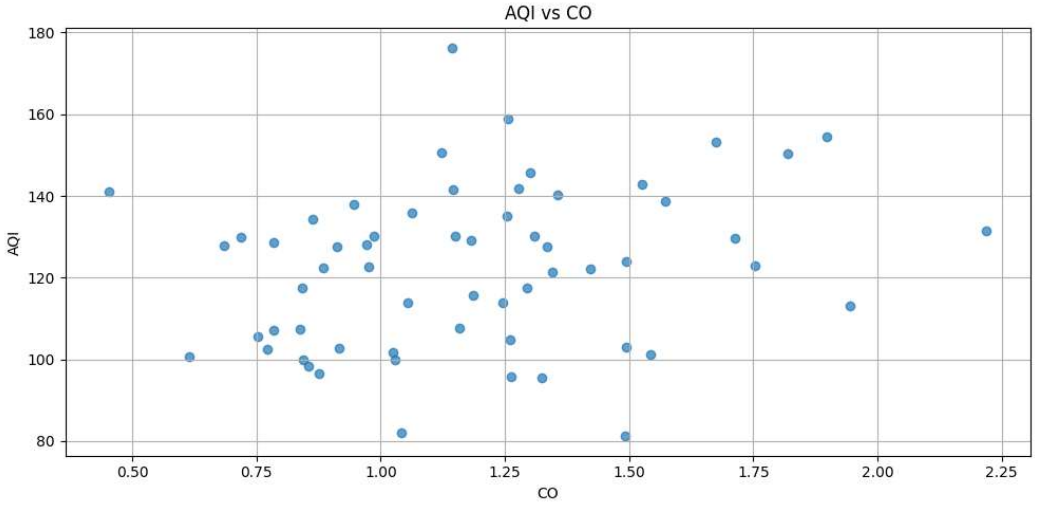


Figure 17: AQI vs CO (scatter plot), correlation = 0.235

What the diagram represents: A weak positive relationship; the cloud rises slightly but with a lot of scatter, so CO explains little of the AQI.

Part B: Top 5 questions an examiner can ask

Q1. What is AQI and what are PM2.5, PM10 and CO?

Ans: AQI (Air Quality Index) is a single number summarising how polluted the air is (higher = worse). PM2.5 and PM10 are particulate matter smaller than 2.5 and 10 micrometres; PM2.5 is more dangerous because it enters the lungs and blood. CO is carbon monoxide, a toxic gas from vehicles and burning fuel.

Q2. Why convert the Date column using pd.to_datetime()?

Ans: The CSV stores dates as text (object). Converting gives datetime values so we can sort chronologically, plot a proper time axis, and use .dt features such as to_period('M') to group by month.

Q3. Which chart would you use to show a trend, a comparison, a distribution and a relationship?

Ans: Trend over time: line chart. Comparison between categories or months: bar chart. Distribution: histogram and boxplot. Relationship between two variables: scatter plot (plus correlation).

Q4. How do you read a boxplot? What is IQR?

Ans: The box shows Q1 to Q3, the line inside is the median, whiskers extend to the smallest and largest non-outlier values and dots beyond them are outliers. IQR = Q3 - Q1 (here about 135.3 - 105.3 = 30). In our plot the median is 123.5 and there are no outliers.

Q5. What did the scatter plots and correlation values tell you?

Ans: AQI vs PM2.5 has r = 0.958 (very strong positive), AQI vs CO r = 0.235 (weak) and AQI vs PM10 r = 0.117 (very weak). So PM2.5 is the dominant pollutant behind AQI in this dataset. Remember that correlation does not mean causation.

Practical 5: Data Aggregation (Sales Performance by Region)

Aim: Aggregate retail sales data (300 transactions, 4 regions, 4 product categories) using groupby to find total sales by region, the top region, and sales by region and product category, and visualise the results with bar and pie charts.

Part A: Code explained line by line, with diagrams

Cell 1: Imports and settings

Code line	Explanation
`import pandas as pd / import numpy as np / import matplotlib.pyplot as plt`	Data handling, numeric library and plotting.
`plt.rcParams[...]`	Default figure size (10 x 5) and grid on.

Cell 2: Loading the data

Code line	Explanation
`try: df = pd.read_csv("Retail_Sales_Data.csv") / except FileNotFoundError / else`	Loads the file with error handling, exactly as in Practical 4.
`display(df.head())`	Shows columns: Transaction_Date, Region, Product_Category, Quantity, Sales_Amount.

Cell 3: Exploring the data

Code line	Explanation
`df.shape`	(300, 5).
`df.dtypes / df.isna().sum()`	Types are correct and there are no missing values.
`df.describe(include="all").T`	Quantity ranges 1-9 (mean 5.02); Sales_Amount ranges 106 to 4,994 (mean 2,536). Region has 4 unique values and West is most frequent (85 rows).

Cell 4: Finding the required columns automatically

Code line	Explanation
`def find_col(columns, keywords):`	Helper function that returns the first column whose name contains any of the keywords.
`x = c.lower().replace(" ", "_")`	Normalises each column name to lower case with underscores.
`if any(k in x for k in keywords): return c`	any() is True if at least one keyword occurs in the name; then that column is returned.
`return None`	If nothing matched, returns None.
`region_col = find_col(df.columns, ["region"])`	Finds 'Region'.
`sales_col = find_col(df.columns, ["sales_amount","sales","amount","revenue"])`	Finds 'Sales_Amount'.
`category_col = find_col(df.columns, ["product_category","category","product"])`	Finds 'Product_Category'.
`df[sales_col] = pd.to_numeric(df[sales_col], errors="coerce")`	Makes sure sales are numeric.

Cell 5: Total sales by region (bar chart)

Code line	Explanation
`region_sales = df.groupby(region_col)[sales_col].sum().sort_values(ascending=False)`	Groups the 300 rows by region, adds the sales in every group and sorts from highest to lowest. Result: North 229,750.53; West 202,726.49; South 165,114.59; East 163,291.77.
`display(region_sales.to_frame("Total Sales"))`	Turns the Series into a one-column table named Total Sales.
`region_sales.plot(kind="bar")`	Draws the bar chart directly from the Series. xticks(rotation=0) keeps region names horizontal.

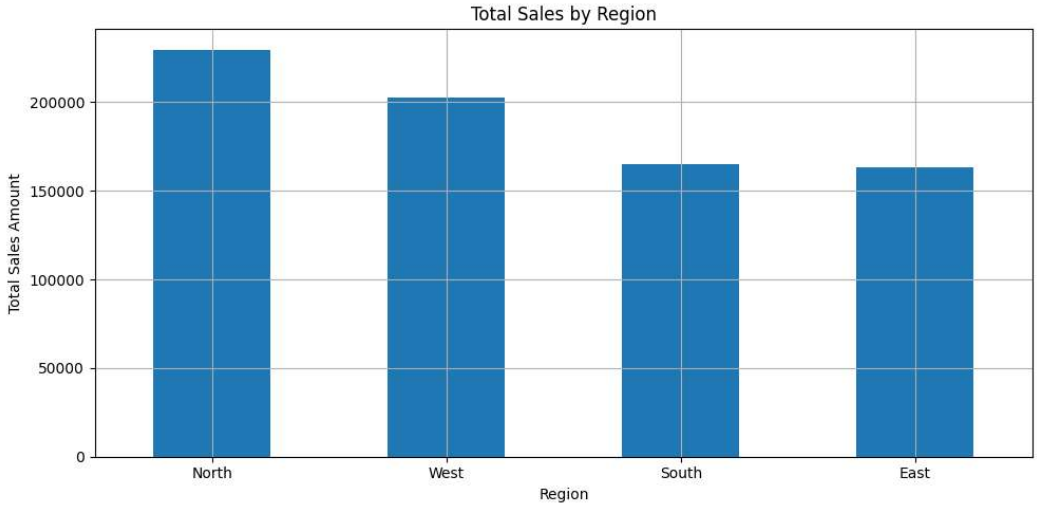


Figure 18: Total sales by region (bar chart)

What the diagram represents: Bars are ordered from highest to lowest: North is clearly the top region (about 229.8 thousand), followed by West (about 202.7 thousand), then South (about 165.1 thousand) and East (about 163.3 thousand), which are almost equal.

Cell 6: Sales distribution (pie chart)

Code line	Explanation
`region_sales.plot(kind="pie", autopct="%1.1f%%", startangle=90)`	Pie chart of each region's share. autopct prints the percentage on every slice (one decimal place) and startangle=90 starts the first slice at the top.
`plt.ylabel("")`	Removes the unnecessary y-label that pandas adds to pie charts.

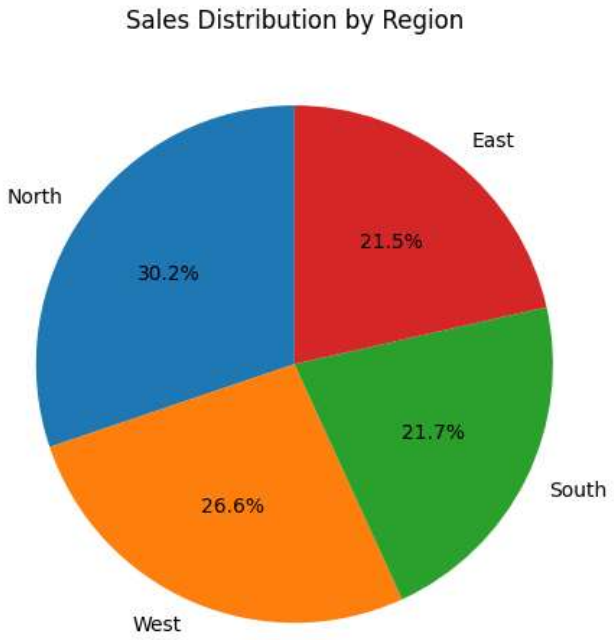


Figure 19: Sales distribution by region (pie chart)

What the diagram represents: A pie chart shows each part as a share of the whole (total sales about 760.9 thousand). North 30.2 %, West 26.6 %, South 21.7 % and East 21.5 %. North and West together generate about 57 % of the sales.

Cell 7: Top-performing regions

Code line	Explanation
`top_regions = region_sales.head()`	Takes the first rows (default 5) of the sorted Series, i.e. the best regions.
`region_sales.index[0]`	The index label of the first (largest) entry gives the best region name: North.
`print(f"Highest-sales region: {region_sales.index[0]}")`	Prints the answer using an f-string.

Cell 8: Region x Product category table

Code line	Explanation
`df.groupby([region_col, category_col])[sales_col].sum()`	Groups by two columns (region and category) and sums sales, giving a multi-level (hierarchical) index.
`.unstack(fill_value=0)`	Moves the inner index level (category) into columns, creating a pivot-style table with regions as rows and categories as columns; missing combinations are filled with 0.

Cell 9: Stacked bar chart

Code line	Explanation
`region_category_sales.plot(kind="bar", stacked=True, figsize=(10, 5))`	One bar per region; the categories are stacked on top of each other so the bar height equals the region's total sales.
`plt.legend(title="Product Category", bbox_to_anchor=(1.02, 1), loc="upper left")`	Puts the legend outside the plot, at the upper left of the anchor point, so it does not cover the bars.

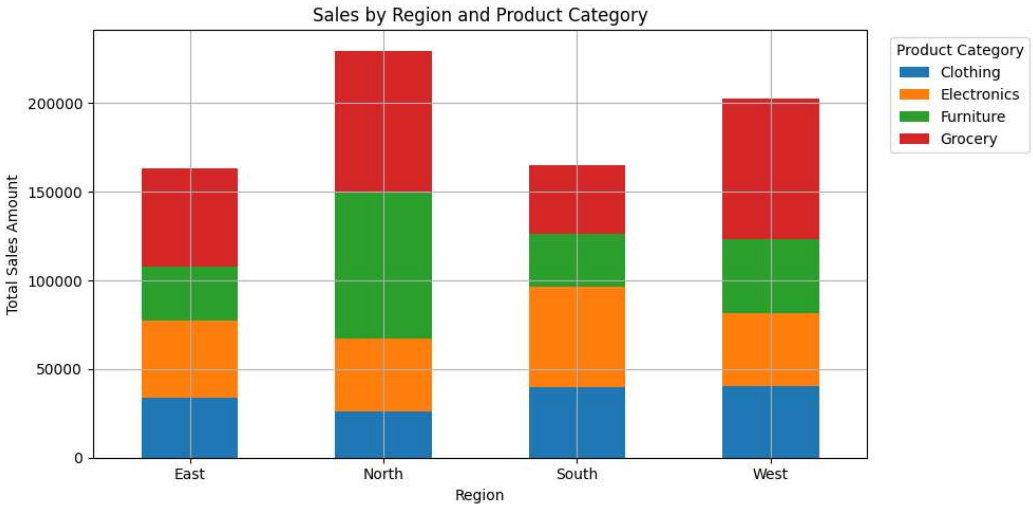


Figure 20: Sales by region and product category (stacked bar chart)

What the diagram represents: Shows both the region total and its category mix. North's bar is mainly Furniture (about 83.0 thousand) and Grocery (about 79.3 thousand), while Clothing is small. West is led by Grocery (about 79.5 thousand). Grocery is the biggest category in East and West.

Cell 10: Grouped bar chart

Code line	Explanation
`region_category_sales.plot(kind="bar", stacked=False, figsize=(10, 5))`	stacked=False places the categories side by side within each region (grouped/clustered bars) so individual values can be compared directly.

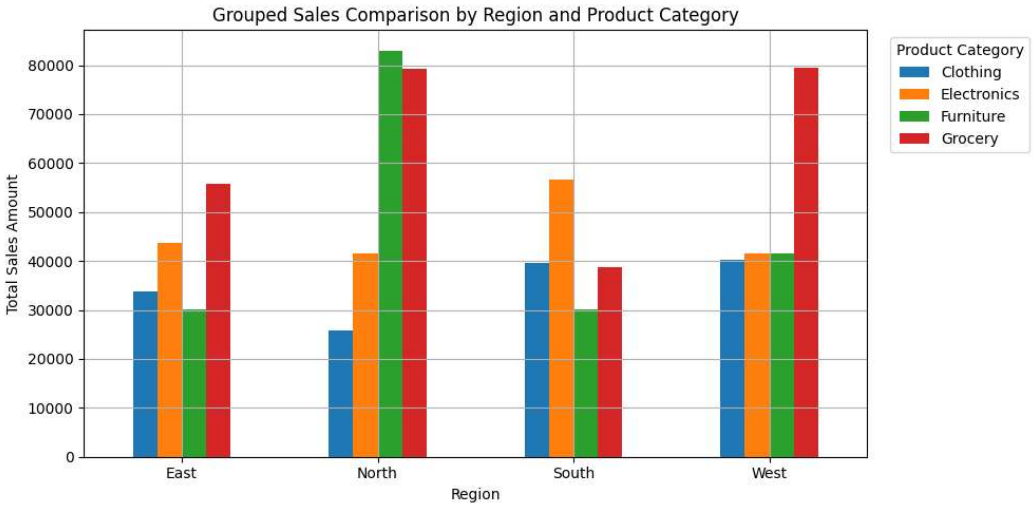


Figure 21: Grouped sales comparison by region and category (grouped bar chart)

What the diagram represents: Easier to compare individual categories across regions. North has the highest Furniture sales, South has the highest Electronics sales (about 56.7 thousand), and Grocery is strongest in West and North (about 79 thousand each). Stacked charts show totals best, grouped charts show individual comparisons best.

Part B: Top 5 questions an examiner can ask

Q1. What is data aggregation? Name some aggregation functions.

Ans: Aggregation summarises many rows into a single value per group. Common functions: sum, mean, median, count, min, max and std, applied with groupby() or agg().

Q2. Explain the groupby split-apply-combine mechanism using your code.

Ans: df.groupby('Region') splits the data into 4 groups, ['Sales_Amount'].sum() applies the sum to each group, and pandas combines the 4 results into a Series (North 229,750.53 and so on).

Q3. What does unstack() do?

Ans: It pivots the innermost index level into columns. groupby([region, category]).sum() gives a Series with two index levels; unstack() turns the categories into columns, making a Region x Category table that is easy to plot as a stacked or grouped bar chart.

Q4. Stacked bar vs grouped bar vs pie chart: when to use which?

Ans: Stacked bar: total per group plus its composition. Grouped bar: compare individual values across groups side by side. Pie: share of each part in a whole, best with few categories. Bar charts are better than pie when exact comparison matters.

Q5. Which region performed best and what is the insight?

Ans: North with total sales of 229,750.53 (30.2 % of all sales), followed by West. North's strength comes from Furniture (about 83 thousand) and Grocery (about 79 thousand). East has the lowest sales (163,291.77), so it may need promotions, especially for Clothing and Furniture.